Task 1: Instagram Bio Prompt & Result
System Prompt:
"Act as a creative social media manager. Write 3 catchy, engaging Instagram bios for a computer science college student. Include relevant emojis, a touch of humor, and a call-to-action line. Keep each under 150 characters."
Generated Result:
1. Coding my way through college, one caffeine cup at a time. | CS '28 |Check out my latest project! [Link]
2. Building app by day, dreaming in binary by night. | University life | DM for collabs!
3. Just a college student trying to fix bugs in my code and life. | Powered by espresso. | Let’s connect!

Task 2: Python RAG Simulation Script
Here is a Python script that simulates a Retrieval-Augmented Generation (RAG) workflow using basic file reading and keyword matching.
python

```
def simulate_rag(question, stats_file_path):
    # 1. Retrieval Phase: Search for relevant lines in the text file
    retrieved_context = []
    keywords = question.lower().split()

    try:
        with open(stats_file_path, 'r') as file:
            for line in file:
                # Check if any keyword matches the text line
                if any(keyword in line.lower() for keyword in keywords):
                    retrieved_context.append(line.strip())
    except FileNotFoundError:
        return "Error: Stats file not found."

    # Combine retrieved lines into a single context string
    context = " ".join(retrieved_context) if retrieved_context else "No specific stats found."

    # 2. Generation Phase: Combine question and context to produce the answer
    if retrieved_context:
        answer = f"Based on the retrieved IPL data ({context}), here is the answer to your question: {question}"
    else:
        answer = f"I couldn't find matching data in the file, but to answer your question: '{question}' — please update the stats file."

    return answer

# Example Usage (Make sure 'ipl_stats.txt' exists with lines like "Virat Kohli scored 973 runs in IPL 2016.")
# print(simulate_rag("How many runs did Virat Kohli score in 2016?", "ipl_stats.txt"))
```

Use code with caution.

Task 3: Real-World Scenarios (Zomato, Flipkart, Paytm)

- Prompt Engineering (Zomato):
  - Scenario: Generating catchy, personalized notification alerts based on food items left in a cart.
  - Reasoning: Fast implementation. No external database search needed. Requires only creative text formatting.
- Retrieval-Augmented Generation (Flipkart):
  - Scenario: A customer support bot answering questions about live delivery status and return policies.
  - Reasoning: Keeps answers accurate. Fetches real-time tracking data directly from internal logistics databases. Prevents AI hallucinations.
- Fine-Tuning (Paytm):
  - Scenario: Training a highly secure internal AI model to detect complex financial fraud patterns and audit logs.
  - Reasoning: Specialized domain knowledge. Requires strict adherence to specific regulatory language and proprietary data formatting.

Task 4: Comparison Table

Criteria	Prompt Engineering	Retrieval-Augmented Generation (RAG)	Fine-Tuning
Setup Cost	Low (essentially free)	Medium (database & vector infrastructure)	High (compute, GPU, and expert labor)
Time to Deploy	Minutes to hours	Days to weeks	Weeks to months
Maintenance Effort	Very low (tweak text)	Medium (updating databases)	High (periodic retraining cycles)
Ideal Use Cases	General writing, summaries	Dynamic data lookup, customer support	Niche domains, strict tone formatting

Task 5: BookMyShow Chatbot Recommendation

The Best Approach: Retrieval-Augmented Generation (RAG)

Final Decision Reasoning:

- Real-time Updates: Movie schedules, seat availability, and showtimes change by the minute. RAG directly pulls this live data.
- Hallucination Control: Prompt engineering alone cannot guess what movies are playing tonight.
- Cost Efficiency: Fine-tuning is too expensive and cannot handle rapidly changing daily data updates effectively.